

+ 26

Ensuring Code Quality in the Age of AI

More Code, Less Engineering!

PETER MULDOON



20
26



Bloomberg

Engineering

Ensuring Code Quality in the Age of AI

More Code, Less Engineering!

CppCon
Sep 15th, 2026

Peter Muldoon
Engineering Lead, TickerPlant

TechAtBloomberg.com

Who Am I



- **Starting using C++ professionally in 1991**
- **Professional Career**
 - **Systems Analyst & Architect**
 - **21 years as a consultant**
 - **15+ years in Bloomberg**
- **Conference talks focused on practical Software Engineering**
 - **Based in the real world**
 - **Take something away or change perspective**

Preamble on AI

AI is changing the world and represents a ton of opportunity for getting more accomplished.

However, problems arise when *accelerating* code production without *strengthening and adapting* the engineering systems that *verify and maintain* that code.

This talk looks at the evidence of what is happening and what engineering organizations can do about it.

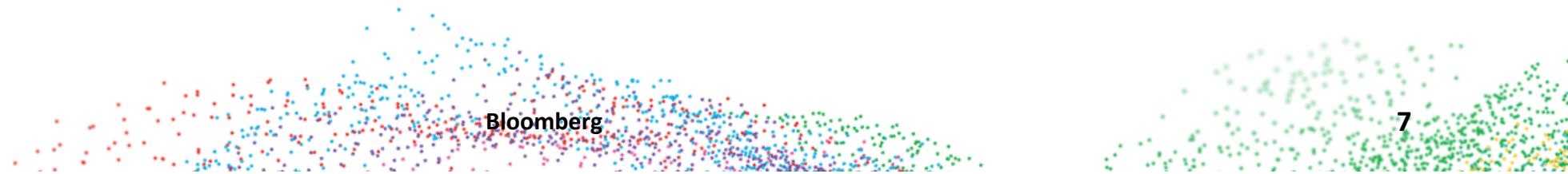
What is an LLM?

How does it work?

Understanding AI : How it works

LLMs are word guessers - Guess the next word(Its probability) and feed it back as its next input iteratively.

Training consists of comparing its guesses to the correct answer and altering internal weights to move it towards the perfect answer.



Understanding AI : How it works

New Orleans is famous for ? :

Temperature	= 1
Mardi	45%
jazz	28%
Creole	17%
Bourbon	10%

Text
↓
Tokens
↓
Numerical representations
↓
"What concepts are relevant to these tokens in this context?"
↓
New Orleans + famous-for relationship
↓
High activation of related concepts
↓
Mardi Gras / jazz / Creole food / French Quarter / ...
↓
Probability distribution over next token
↓
Generate one token



Understanding AI : How it works

New Orleans is famous for ? :

New Orleans is famous for **Jazz** (Pass 1)

New Orleans is famous for Jazz, **Mardi** (Pass 2)

New Orleans is famous for Jazz, Mardi **Gras** (Pass 3)

Temperature	= 1
Mardi	45%
jazz	28%
Creole	17%
Bourbon	10%

New Orleans is especially famous for:

- **Jazz** — widely regarded as the birthplace of jazz.
- **Mardi Gras** — parades, costumes, floats, beads, and huge celebrations.
- **Creole and Cajun food** — gumbo, jambalaya, étouffée, red beans and rice, po'boys.

ChatGPT can make mistakes. Workspace data isn't used to train models.

Understanding AI : How it works

LLMs are word guessers - Guess the next word(its probability) and feed it back as its next input iteratively.

Training consists of comparing its guesses to the correct answer and altering internal weights to move it towards the perfect median answer.

The intelligence property is emergent i.e. not programmed in and its behavior is non-deterministic.

The Issue : *This breaks one of the fundamental assumptions of traditional engineering*

The same input should produce the same output, and that output can be tested once and trusted from then on.

Understanding AI : Where it naturally fits

Great for

- **Brainstorming**
 - *“What are the possible ideas, options, risks or approaches we should consider?”*
- **Rapid Prototyping**
 - *“Can we build something that demonstrates this idea?”*
- **Experimentation**
 - *“What is the best way to solve this?”*
 - *“How does this technology or technique work?”*
- **Just-in-time technical lookup**
 - *“How do I do this specific thing for this immediate case?”*
- **Throwaway utility work**
 - *“Useful output, but intentionally temporary?”*
- **Rapid analysis for a general gist**
 - *“What does this data appear to be telling us?”*

Understanding AI : Where caution is required

Caution when used for

- **Production-critical code**
 - *“Can we safely rely on this in production?”*
- **Safety-critical systems**
 - *“Could an error cause physical harm?”*
- **Security-sensitive work**
 - *“Could getting this wrong create a vulnerability?”*
- **Regulatory / compliance work**
 - *“Does this need to be demonstrably correct against a rule or obligation?”*
- **Irreversible or difficult-to-reverse actions**
 - *“If this is wrong, can we easily undo it?”*
- **Correctness is difficult to inspect:**
 - *“Can a reviewer realistically tell whether this is right?”*

When used for the above applications, Engineering discipline must adapt

Understanding AI : How to fit

Low-rigor LLM work	Dicey LLM work
Easy to redo	Hard to reverse
Easy to inspect	Hard to verify
Small/local impact	Large blast radius
Directional answer is sufficient	Authoritative / Evidentiary answer required
Disposable	Long-lived
No serious consequence	Material consequence : physical, financial, legal, security, or operational harm

AI in the News?

Lessons outside of Software Engineering

Publishing

N.E Global News : Book publishing's AI panic is here. And nobody knows what to do about it

Across 2025 and 2026, major publishing houses like *Hachette Book Group* and various international literary agencies took (such as the anticipated horror the ***extreme step of completely canceling multi-million dollar book deals*** novel *Shy Girl* and crime thriller *Call Me, I'll Hide the Body*) after software scans and online sleuths flagged that portions of the texts heavily relied on uncredited AI text generation and text smoothing

Given the economic realities of publishing houses, there's also the ethical question of whether ***publishers would one day use AI to replace editors/authors***

Law

Sullivan & Cromwell law firm apologizes for AI ‘hallucinations’ in court filing : 04-18-2026

One of Wall Street’s most prominent law firms, apologized to a federal bankruptcy judge after filing a brief with more than 36 instances of **hallucinated legal summaries, fake case citations, and misquoted laws** created by a generative tool

A secondary internal ***human review process completely failed to catch the errors*** before submission. Opposing counsel quickly cross-referenced the text and blew the whistle.

U.S. judges have sanctioned lawyers in dozens of cases after attorneys used AI for [legal research](#) and drafting without fully vetting the results. Lawyers are not prohibited from using AI but are ***ethically bound*** to ensure the accuracy of court submissions.

Pharmaceuticals

Clinical trial vanguard - \$7 Billion Into AI Drug Discovery and Zero Approved Drugs

An [industry analysis](#) found that roughly \$7 billion invested in AI-driven drug discovery as of Jan 2026 has yet to produce a single drug approved by the Food and Drug Administration.

The first fully AI-discovered and AI-designed molecule to publish positive Phase IIa efficacy results is a genuine milestone. The cost of converting candidates into drugs remains ***approximately \$2.6 billion per approved product, with only about 12% of molecules entering clinical trials ever reaching patients.***

Three of the most consequential drugs (*Lipitor, Gleevec, and Cyclosporine*) in modern pharmacology **would have been screened out by the same logic now being automated at billion-dollar scale.**

"A faster discovery engine attached to an unreformed development engine does not accelerate drug approval. It accelerates congestion" – Milad Alucozai

Manufacturing

Ford Motor - AI Quality Cameras Backfire, "Greybeards" Rehired (06-29-2026)

Ford deployed **900 AI-powered cameras** across its manufacturing plants to detect quality defects, alongside broader automation initiatives intended to reduce reliance on experienced human technicians.

Ford has since **rehired approximately 350 veteran technicians** to train both younger workers and the AI tools themselves, effectively acknowledging that the **institutional knowledge of experienced workers** could not be replicated by the systems deployed to replace them. [\(1\)](#)

Charles Poon, its vice president of vehicle hardware engineering, the auto company had “**mistakenly**” assumed that feeding its design requirements into AI would *by itself* produce a high-quality product.

Trends

Broad Corporate Trend - The AI Math To Do Before Laying Off Employees (7-30-2026)

A Forbes analysis found that roughly **one-third of companies that laid off employees due to AI automation have already hired people back to fill some of those vacated roles**, with approximately half expected to make similar moves by 2027. The pattern suggests that many organizations ***underestimated*** the complexity of the tasks they sought to automate and the ***hidden costs of losing institutional knowledge***. [\(3\)](#)

Resistance

A Hidden Cost of Forced AI Adoption

New research from Apollo Global Management shows the technology's earliest measurable damage isn't job losses, but smaller paychecks.

A Fortune survey found that **nearly one-third of workers admit to actively sabotaging their company's AI systems**, with wage suppression cited as a key motivator.

There is also a lot of AI shame going on: 13% admitted they've faked AI use - appearing to use a tool while doing the task manually - and only 6% believe their managers accurately understand how often employees actually use the tools they've rolled out [\(7\)](#)

The Cost of being Wrong

When mistakes are costly, ***verification trumps speed?***

- Commercial aircraft have flown on autopilot for decades, yet two pilots still sit in every cockpit
- Hospitals that let AI systems suggest diagnoses still keep physicians in the loop, because a mistaken recommendation carries a cost no benchmark captures

In heavily regulated sectors such as transportation, aviation, pharmaceuticals, or financial services - ***safety requirements*** and ***compliance frameworks*** slow technology integration.

Public Service Announcement



**I LOVE AI AND USE IT
A LOT**

Software: The Ideal Environment for AI Adoption

Software engineering faces minimal barriers to AI implementation

Generative AI has *eliminated* the natural bottlenecks that previously controlled ***code quality: human typing speed, debugging time, and implementation complexity***

The real bottleneck in AI deployment is not the *cost of* generating answers, but the ***cost of verifying*** whether those answers are safe, accurate, and accountable enough to deploy

Software + AI : Ideal Fit

Cheaper Faster AI Doesn't Matter If Nobody Trusts It: Len Khodorkovsky [link](#)

The "**Actionability Gap**" refers to the distance between an answer a model can generate and one an institution can activate. While the cost of generation has decreased exponentially, the cost of verification has not."

Software is the one field where that gap has almost closed, and it's instructive precisely because of why.

- Code compiles or it doesn't.
- Tests pass or fail.
- A broken build is caught in seconds, at almost no cost.

"Everywhere else, a wrong answer risks a lawsuit, a license (revocation) or a patient (hospital)."

"Answers are becoming *commodities*. Institutional confidence in them is not."

What is Code Quality?

How to measure it ?

The Question

Was generating *lines of code / commit count* the major impediment to delivering a stable product with new features/capabilities?

To support this, did we have

- Sufficient testing - Correctness of results
- Consistent standard of code review - Maintainability
- Good deployment mechanisms - Efficiently move solutions to production

❖ In the **Age of AI** : Can we quantify the cost of *shipping more* while *caring less* ?

Some Answers



AI Copilot Code Quality

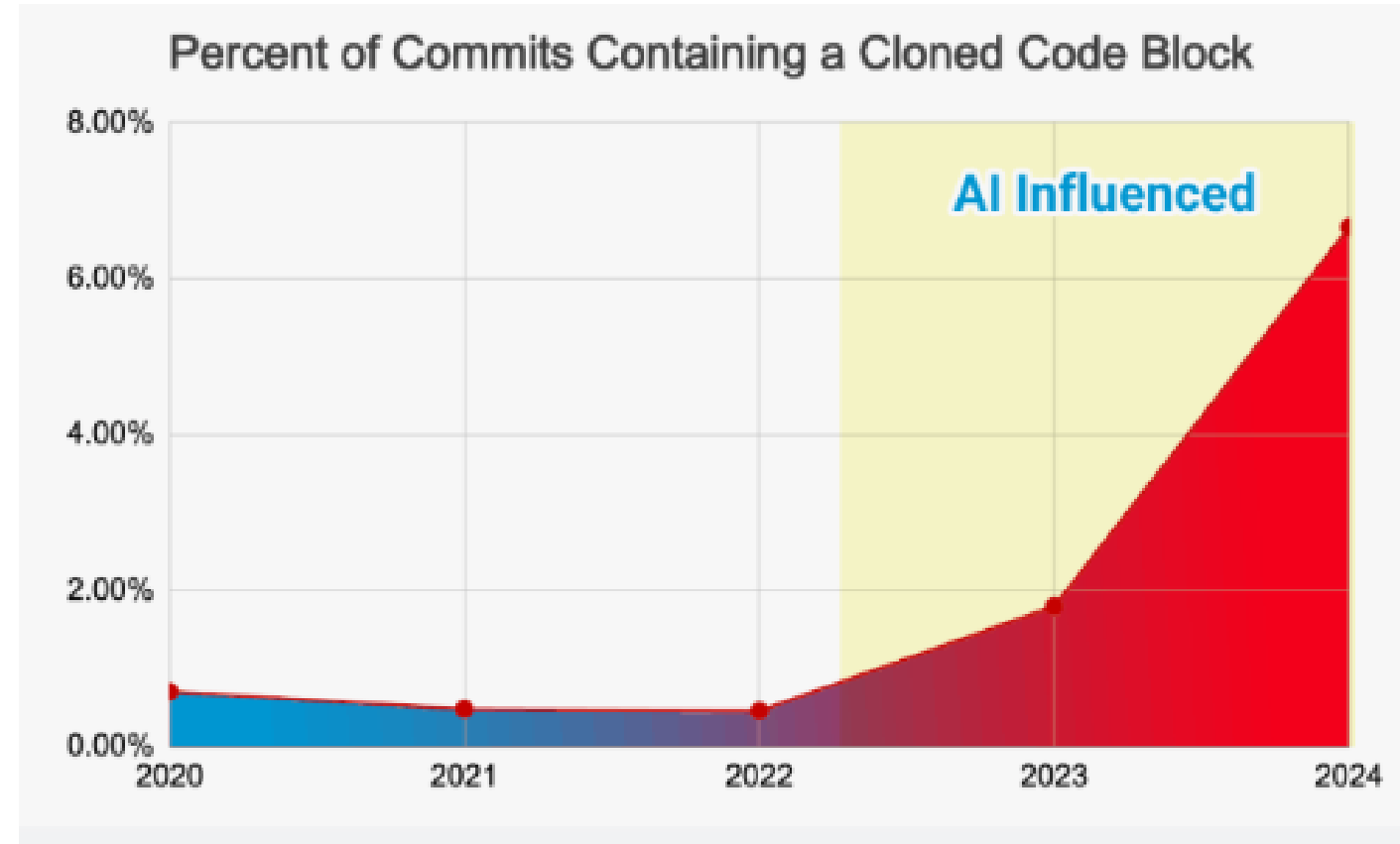
*Evaluating 2024's Increased Defect Rate
via Code Quality Metrics*

211m lines of analyzed code + projections for 2025

AI + Code Duplication

Emerging trends: 4x more code cloning, includes 2025 projections

1. 2024 marked the first time "**copy/paste**" exceeds "**moved**" code for first time in history.
2. Coupled with an 8-fold increase in frequency duplication during 2024



AI + Code Duplication

The Maintainability Gap

AI Code Quality in 2026

Usage has never been higher. Neither have the risks.

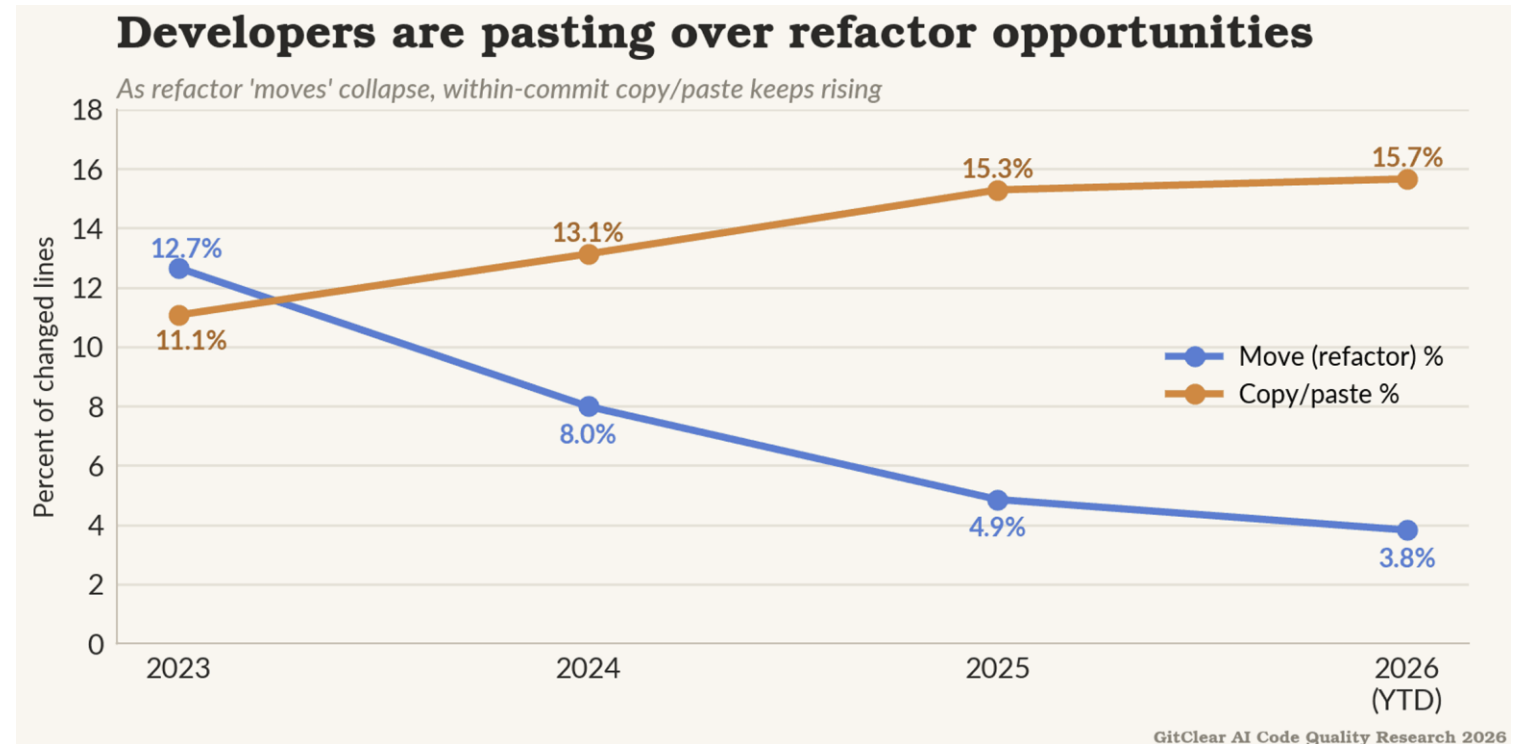
623 million analyzed changes from 2023-2026, tracking eight quality signals as AI authorship reaches record volume.



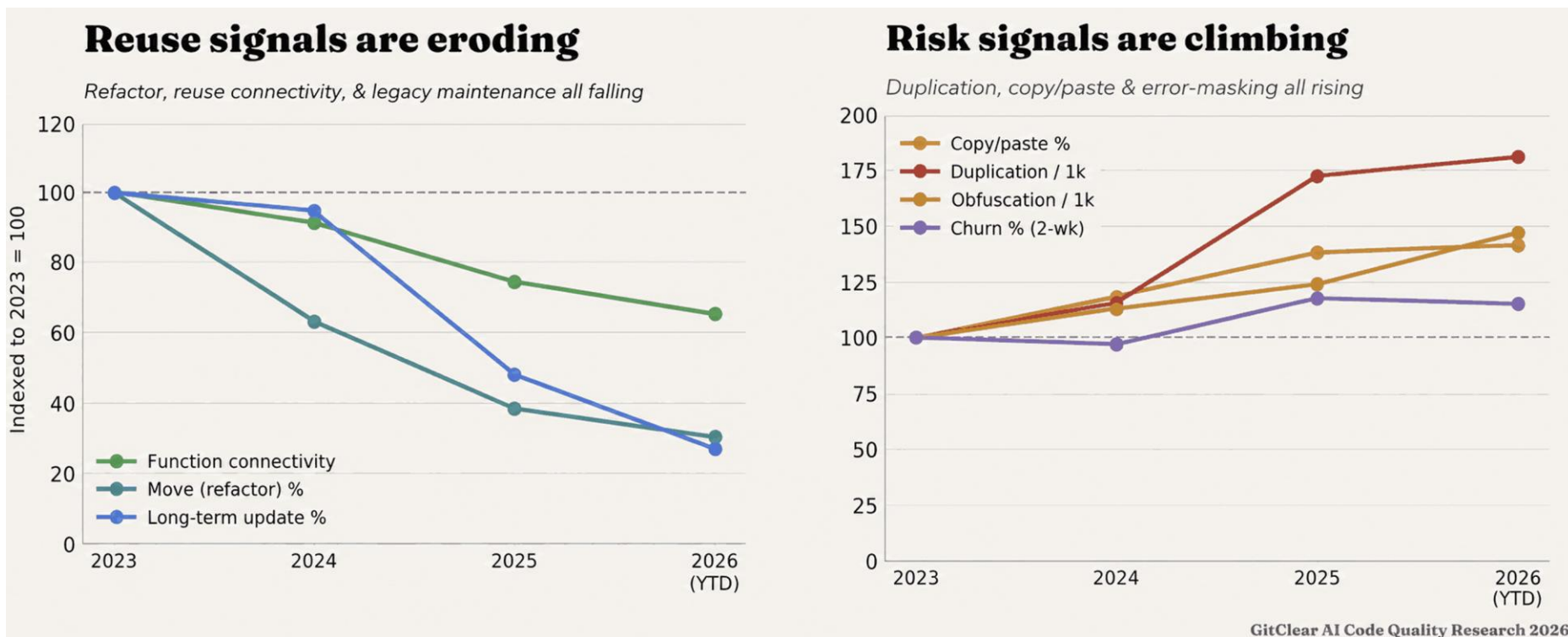
AI + Code Duplication

The fundamental problem

- Adding code to a new file requires almost zero architectural understanding of the repo.
- Refactoring requires understanding the surrounding system well enough to consolidate it safely
- A defect discovered in one copy of a block must be hunted down and fixed in every sibling, and the ones that are missed become latent bugs



AI + Code Duplication



- When it wasn't free to add new lines (pre-2023), 20-25% of all code changes were “moves”
- Healthy, long-lived repositories required developers to periodically return to old code: consolidate it, document it, retire it. That maintenance work is increasingly not happening

Measuring AI Impact



Code Quality Erosion

Developers are doing more. Execution is accelerating.

- Deployment frequency is the exception
- Code churn raises a question about what counts

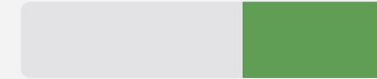
OUTPUT IS UP. NOT ALL OF IT STICKS.

Impacts of high AI adoption on engineering throughput



Epic

A sizable feature or capability, hundreds of tasks



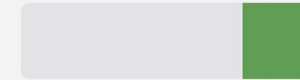
+66.2%

Epics completed per developer



Task

Individual work item, may span multiple PRs



+33.7%

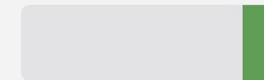
Task throughput per developer

Within this, tasks with an associated PR grew **+210%** per team. Code-specific work accelerated far faster than overall task volume.



Pull Request (PR)

Atomic unit of code change



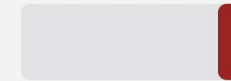
+16.2%

PR merge rate per developer



Deployment

Ships code changes to production



-11.7%

Deployments per week

More code is completing and merging, but fewer deployments are reaching production. The arm that ships is under pressure. DORA metric • 10% of dataset



Code Churn

Lines deleted to lines added per quarter, for merged code



+861%

Code deletion ratio

Telemetry dataset size: 22,000 developers across 4,000 teams
AI Engineering Report 2026 | The Acceleration Whiplash

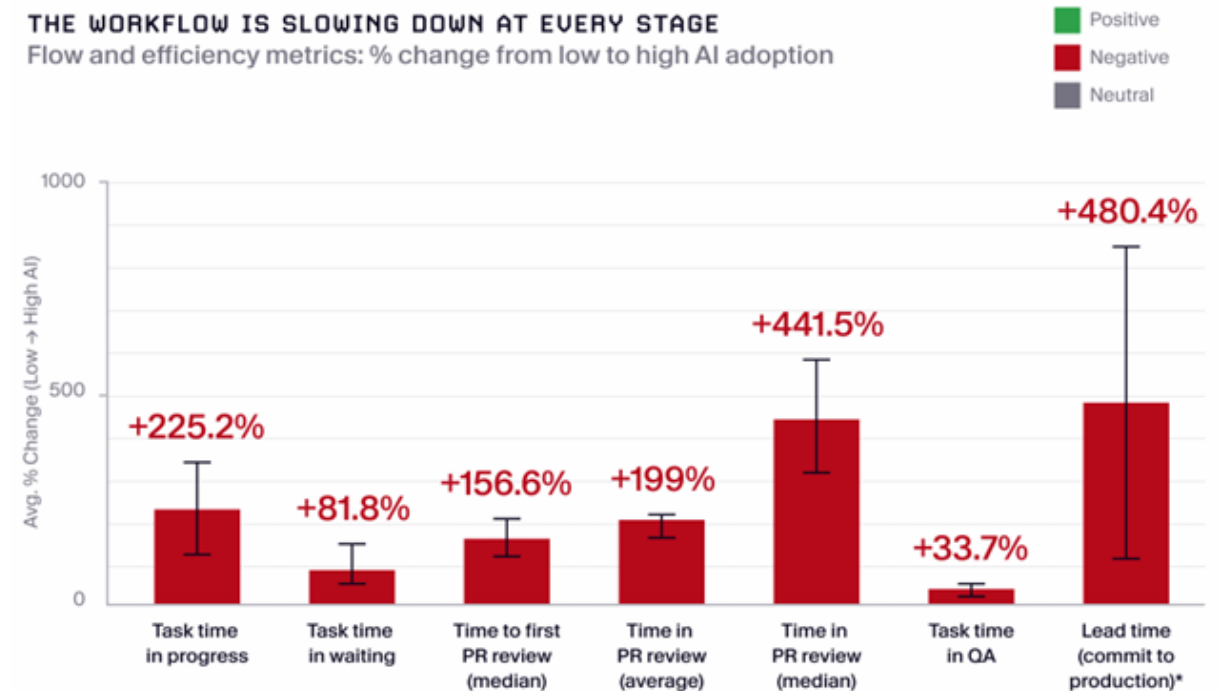


Code Quality Erosion

Code changes are getting larger and more complex. Every structural dimension has grown.

- +51.3% average PR size
- +59.7% average files edited per PR
- +149.9% average files touched per developer per month
- +11.7% average repositories touched per developer per month
- +31% PRs are being merged without any review, human or agentic

THE WORKFLOW IS SLOWING DOWN AT EVERY STAGE
Flow and efficiency metrics: % change from low to high AI adoption



Telemetry dataset size: 22,000 developers across 4,000 teams
AI Engineering Report 2026 | The Acceleration Whiplash

* Measured by 10% of the dataset



Code Quality Erosion

Poor quality code is reaching Production

The code entering production systems is not meeting the bar that engineers once set for themselves.

Incidents have tripled relative to the low AI adoption baseline

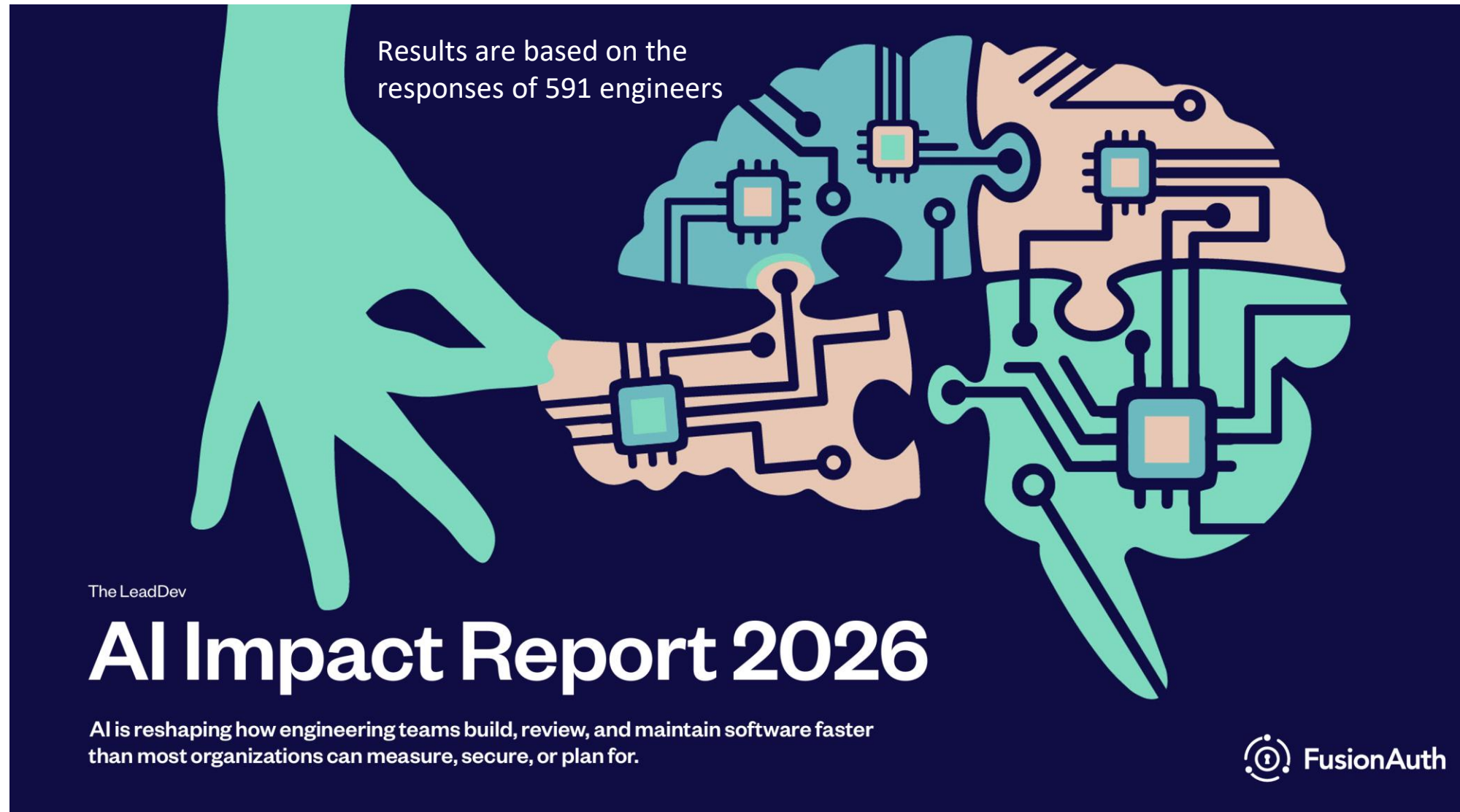


Public Service Announcement



**I LOVE AI AND WANT
IT TO SUCCEED**

Measuring AI Impact

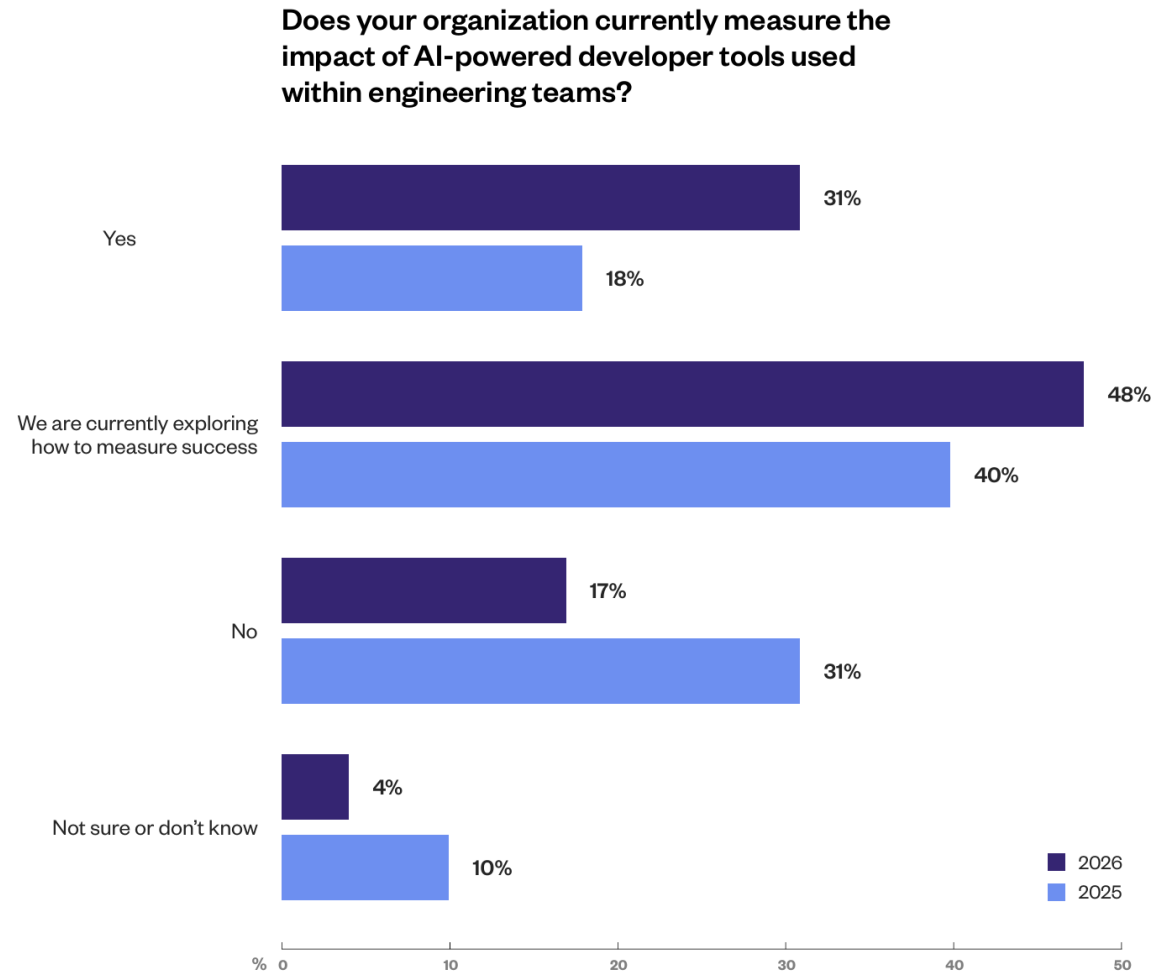


Measurement Gap

The trouble is that almost nobody can back up their assertion of productivity gains with evidence

Only **31%** of organizations currently measure the impact of AI-powered developer tools at all, up from 18% a year ago

Another **48%** say they're currently exploring how to measure success, which is a polite way of saying nobody's built the dashboard yet

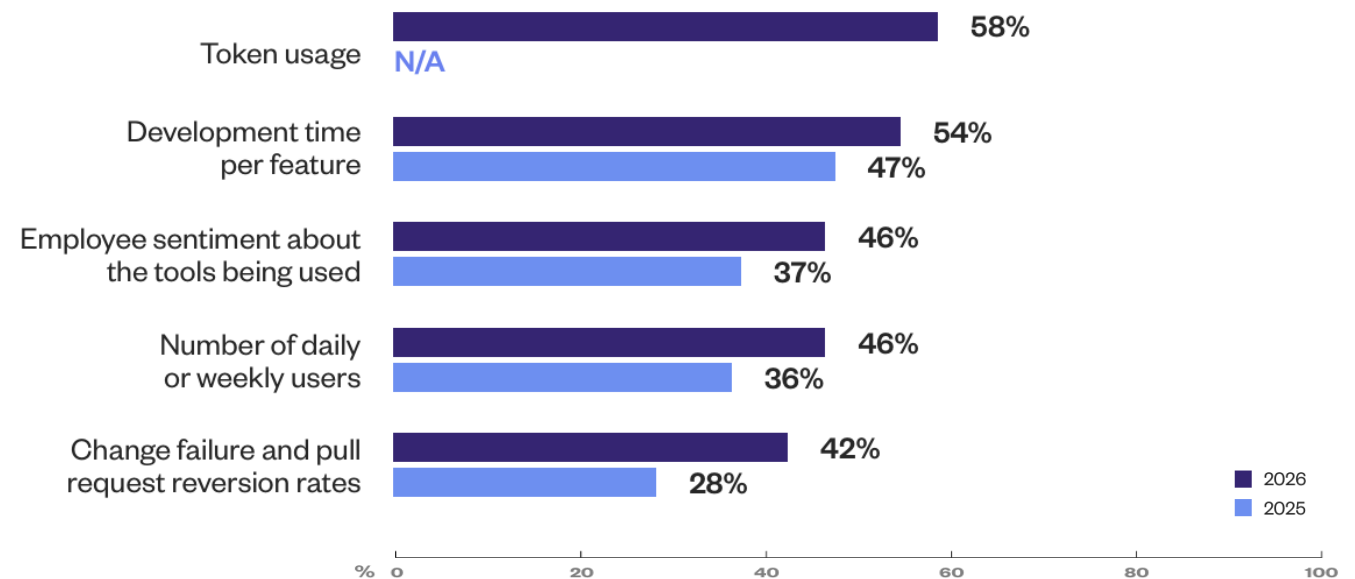


Measurement Gap

68% cited “lack of clear metrics to evaluate AI’s impact on productivity or quality”

- This is the single most common complaint in the whole survey
- Among the 31% who do measure, the metrics are getting more sophisticated without necessarily earning anyone’s trust
- **Token usage is tracked by 58% of measuring organizations**, edging out development time per feature at 54%
- **58% say token usage metrics are ineffective**

If your organization measures the impact of AI-powered developer tools used within engineering teams, what key metrics are tracked?



58% of respondents say token usage metrics are ineffective at gauging real value from AI tools

Measurement Gap

Gautum Mukunda : Are AI deployment targets just another management fad?

“Is evaluating people by the amount they spend, not the output they produce, a good idea?”

Normally no, but sprinkle some AI pixie dust on the question and you get

- *Meta Platforms Inc.* running a leaderboard for employees that handed out titles like “Token Legend” to the heaviest users, who in 30 days consumed more than **70 trillion tokens**
- *Amazon.com Inc.* setting weekly AI-use targets for most of its employees. Engineers naturally began handing AI agents busywork

Both used token burn leaderboards until Meta reversed course and told its people to use less.

- *Uber Technologies Inc.* urging its staff to use AI as much as possible and ***exhausting its entire annual budget for coding tools in four months*** (It has since capped employee usage)

Asked whether all that ***consumption was producing more of the features customers want,***

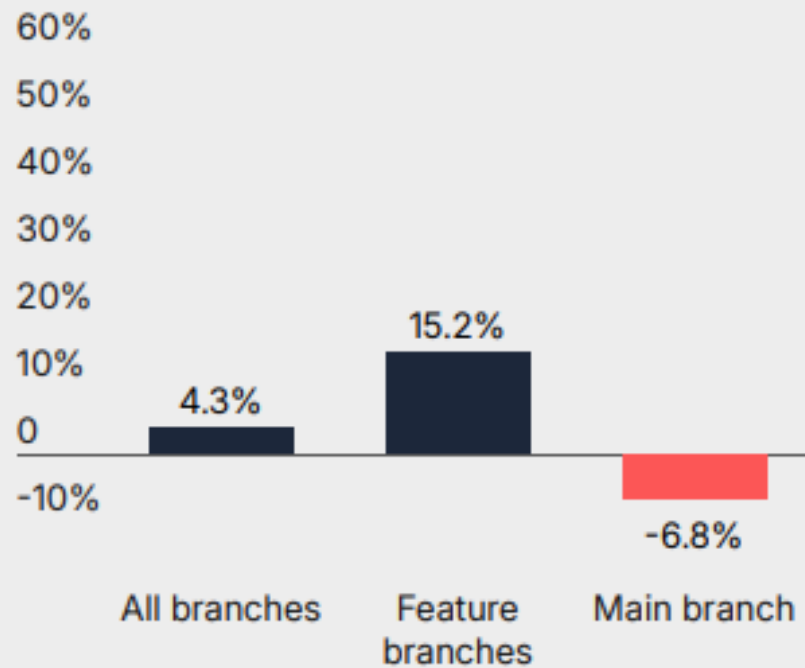
Uber President Andrew Macdonald was candid: “**That link is not there yet.**”

Measuring AI Impact



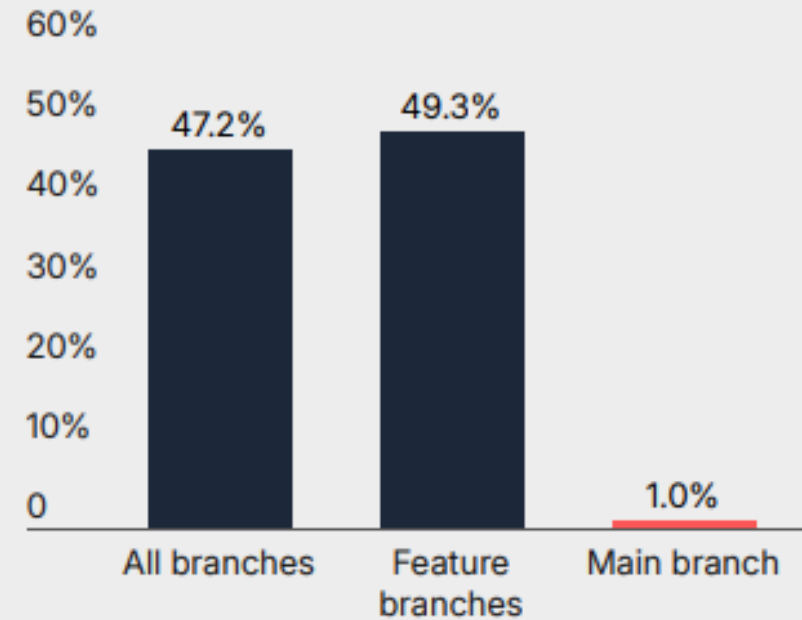
Developer Productivity

Change in throughput for median teams,
by branch



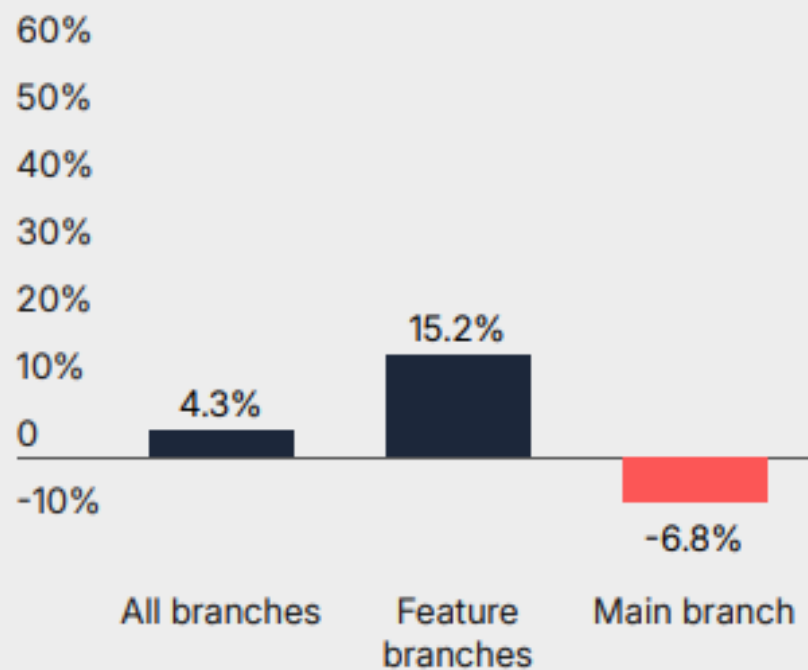
CircleCI | The 2026 State of Software Delivery

Change in throughput for p90 teams,
by branch



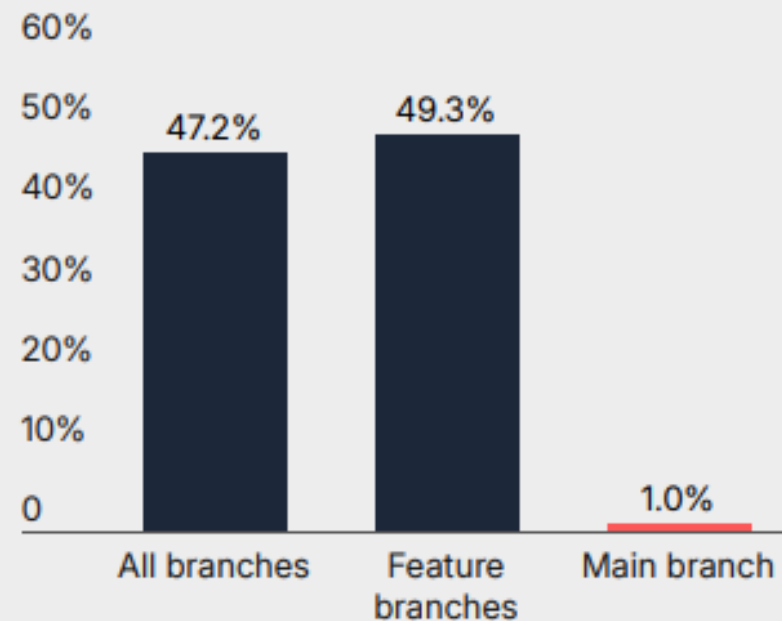
Developer Productivity

Change in throughput for median teams,
by branch



CircleCI | The 2026 State of Software Delivery

Change in throughput for p90 teams,
by branch



Measurement Gap

Companies are driving adoption faster than they can measure value

- Driven on conviction instead of results
- Code churn & Duplication are a worrisome trend
- Productivity Metrics are still a relative unknown area
- Outcomes at the enterprise level have not seen the gains been reported at local level

Public Service Announcement



**I'M STILL PRETTY
OPTIMISTIC ABOUT AI
AND STILL WANT IT TO
CONTINUE USING IT**

Impact of AI

**What does the Human
Developer & Manager
think?**

Code Quality Erosion

Anthropic [Report](#)



Contrary to our initial hypothesis, we did not observe a significant performance boost in task completion in our main study. While using AI improved the average completion time of the task, the improvement in efficiency was not significant in our study.

There is a group of participants who relied on AI to generate all the code and never asked conceptual questions or for explanations. This group finished much faster than the control group.

Other participants in the AI group who asked a large number of queries (e.g., 15 queries), spent a long time composing queries (e.g., 10 minutes), or asked for follow-up explanations, raised the average task completion time.

The erosion of conceptual understanding, code reading, and debugging skills that we measured among participants using AI assistance suggests that workers acquiring new skills should be mindful of their reliance on AI during the learning process.

Code Quality Erosion



Anthropic [Report](#)

Contrary to our initial hypothesis, we did not observe a significant performance boost in task completion in our main study. While using AI improved the average completion time of the task, the improvement in efficiency was not significant in our study.

There is a group of participants who relied on AI to generate all the code and never asked conceptual questions or for explanations. This group finished much faster than the control group.

Other participants in the AI group who asked a large number of queries (e.g., 15 queries), spent a long time composing queries (e.g., 10 minutes), or asked for follow-up explanations, raised the average task completion time.

The erosion of conceptual understanding, code reading, and debugging skills that we measured among participants using AI assistance suggests that workers acquiring new skills should be mindful of their reliance on AI during the learning process.

Code Quality Erosion

Anthropic [Report](#)



Contrary to our initial hypothesis, we did not observe a significant performance boost in task completion in our main study. While using AI improved the average completion time of the task, the improvement in efficiency was not significant in our study.

There is a group of participants who relied on AI to generate all the code and never asked conceptual questions or for explanations. This group finished much faster than the control group.

Other participants in the AI group who asked a large number of queries (e.g., 15 queries), spent a long time composing queries (e.g., 10 minutes), or asked for follow-up explanations, raised the average task completion time.

The erosion of conceptual understanding, code reading, and debugging skills that we measured among participants using AI assistance suggests that workers acquiring new skills should be mindful of their reliance on AI during the learning process.

Code Quality Erosion



Anthropic [Report](#)

Contrary to our initial hypothesis, we did not observe a significant performance boost in task completion in our main study. While using AI improved the average completion time of the task, the improvement in efficiency was not significant in our study.

There is a group of participants who relied on AI to generate all the code and never asked conceptual questions or for explanations. This group finished much faster than the control group.

Other participants in the AI group who asked a large number of queries (e.g., 15 queries), spent a long time composing queries (e.g., 10 minutes), or asked for follow-up explanations, raised the average task completion time.

The erosion of conceptual understanding, code reading, and debugging skills that we measured among participants using AI assistance suggests that workers acquiring new skills should be mindful of their reliance on AI during the learning process.

Code Quality Erosion



Anthropic [Report](#)

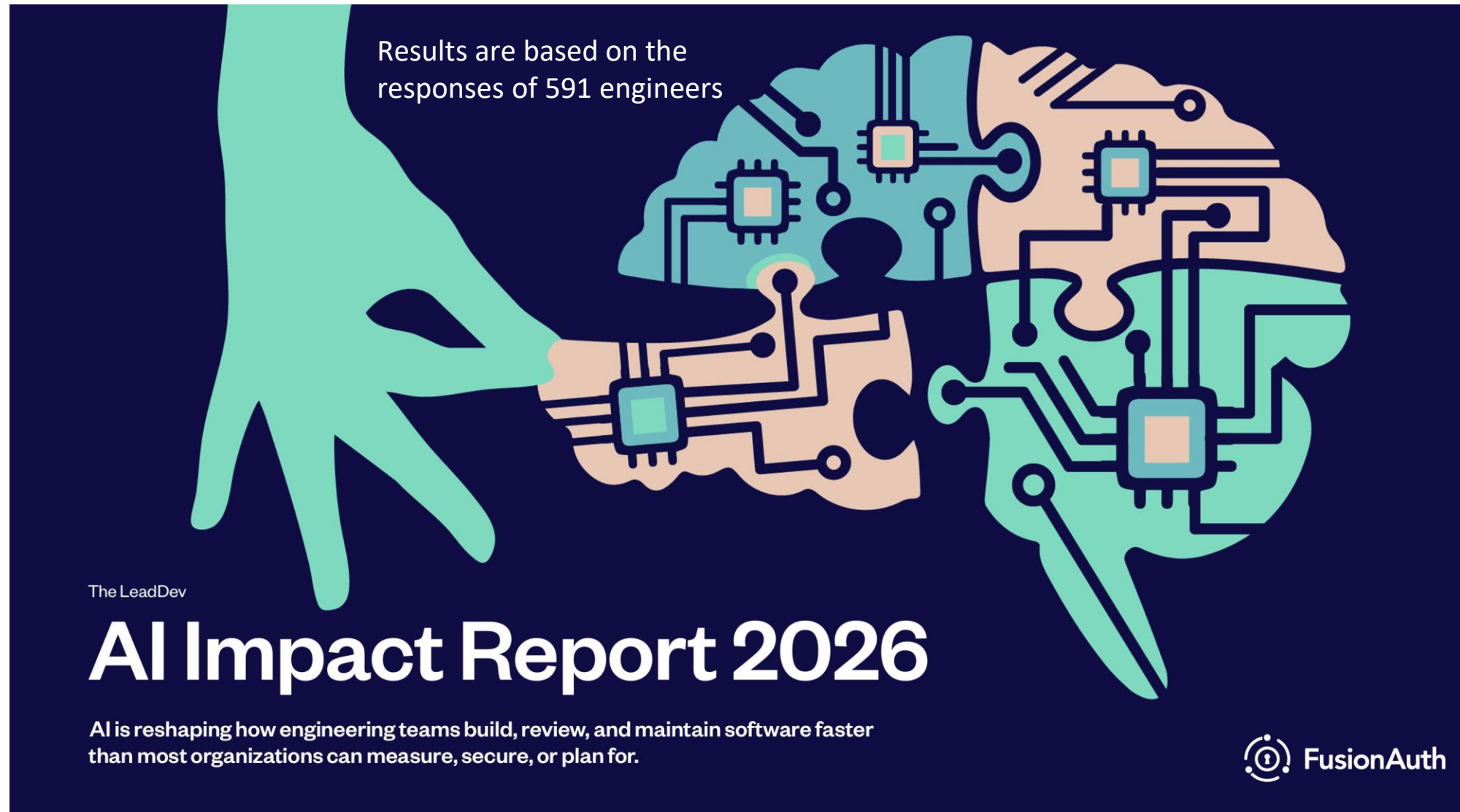
Contrary to our initial hypothesis, we did not observe a significant performance boost in task completion in our main study. While using AI improved the average completion time of the task, the improvement in efficiency was not significant in our study.

There is a group of participants who relied on AI to generate all the code and never asked conceptual questions or for explanations. This group finished much faster than the control group.

Other participants in the AI group who asked a large number of queries (e.g., 15 queries), spent a long time composing queries (e.g., 10 minutes), or asked for follow-up explanations, raised the average task completion time.

The erosion of conceptual understanding, code reading, and debugging skills that we measured among participants using AI assistance suggests that workers acquiring new skills should be mindful of their reliance on AI during the learning process.

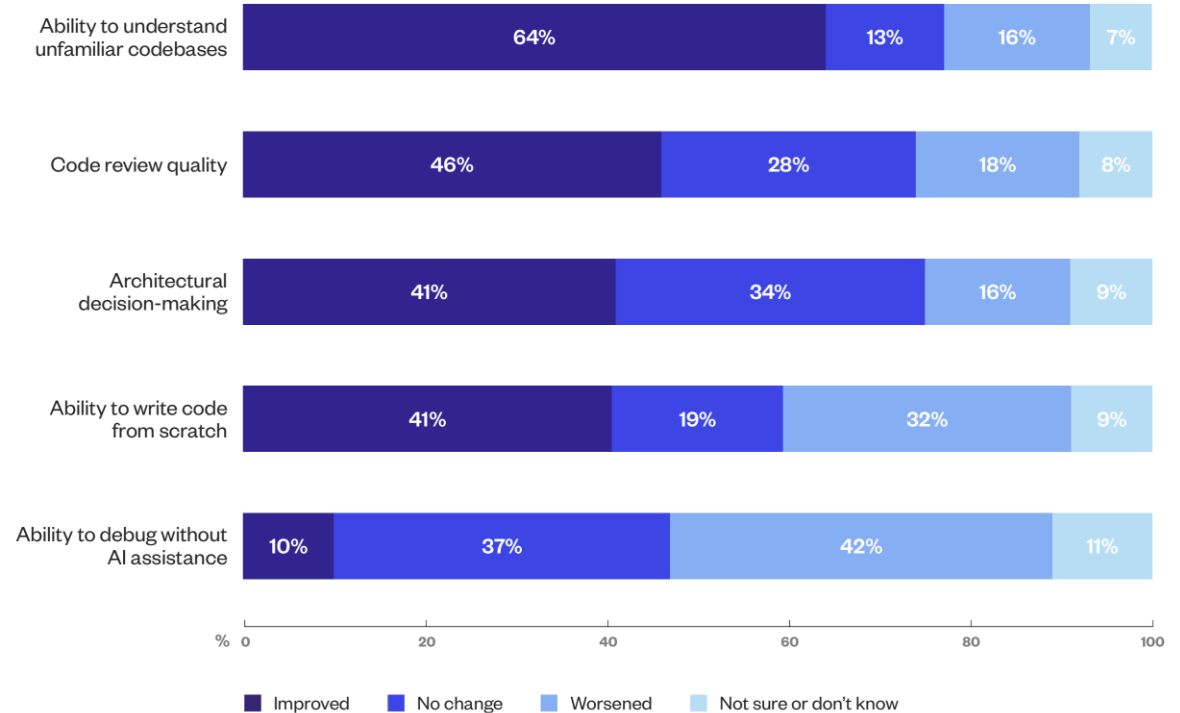
Measuring AI Impact



Skills Shift

- **64%** of respondents believe AI has improved their team's ability to understand unfamiliar codebases against **16%** say it's worsened
- **41%** of respondents believe AI has improved Architectural against **16%** say its worsened
- **42%** of respondents believe overreliance on AI tools has left their teams **less able to debug** without them

In the past 12 months, how has the adoption of AI-powered developer tools impacted the following engineering competencies within your team?



Skills Shift

Looking three years ahead, engineering leaders expect a different set of skills to matter most.

On the rise:

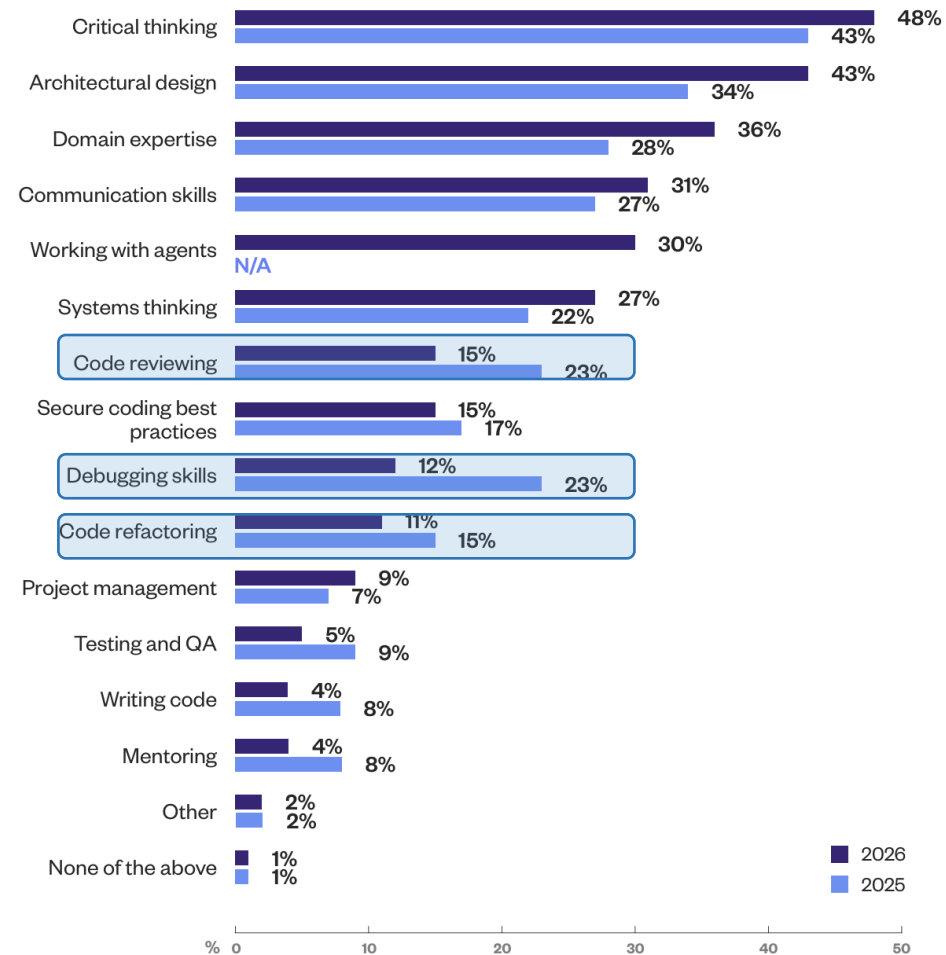
- Critical thinking (48%)
- Architectural design (43%)
- Domain expertise (36%)

On the decline:

- Code reviewing (15%, down from 23%)
- Debugging (12%, down from 23%)
- Code Refactoring (11%, down from 15%)

That's a sharp reversal from where leaders thought the industry's attention would be a year ago

Over the next three years, which areas of engineering competence do you think will be most in demand due to increased use of AI?



Role Measures

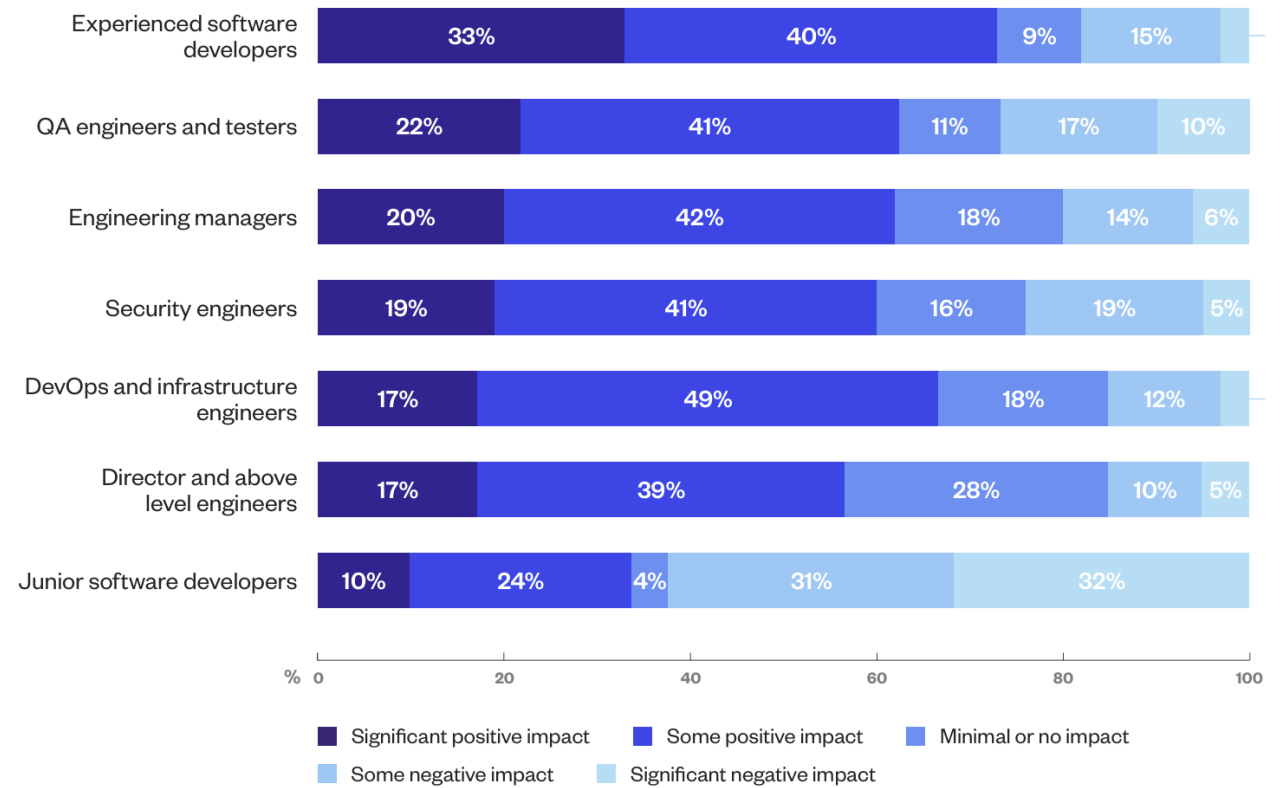
AI-powered developer tools are having a positive impact on

- Experienced software developers (+73%)
- DevOps engineers (+66%)
- QA engineers and testers (63%)
- Engineering managers (62)
- Security engineers (60%).

Junior software developers are the outlier:
34% positive against 63% negative

Ironically some citing *Anthropic* as a counter argument will likely keep hiring junior engineers

What will the impact of AI-powered developer tools and models on the following engineering roles?



Role Summary

Skills in demand are changing with AI adoption (reliance)

- Skills are drifting toward **architecture and critical thinking (a good thing)**
- The ability to **debug without AI/Review/Refactor** quietly slips away (a not-so-good thing)
- What happens to the **junior engineers** who used to be the pipeline for seniors
- Engineering competence with AI eroding deep knowledge

Public Service Announcement



**I'M NOT SO SURE
ABOUT AI. CAN I USE
IT SAFELY?**

AI Debt

Major Types of GenAI Debt - Margaret-Anne Storey

- **Technical Debt:** Code duplication, poorly structured, latent/subtle bugs
- **Cognitive Debt:** The loss of team understanding because developers accept functional code without knowing how it works under the hood.

Battle for Control

Cognitive Delegation: The intentional transfer of part of a thinking task to an external tool or agent while retaining human judgment and accountability for the outcome

- **Cognitive Offloading:** Human judgement steers decision making, AI is a tool which is questioned and the reasoning verified
- **Cognitive Surrender:** The uncritical acceptance of artificial intelligence outputs in place of one's own internal reasoning and judgment

Cognitive Surrender

Researchers Steven D. Shaw and Gideon Nave of the Wharton School have published

Thinking - Fast, Slow, and Artificial: How AI is Reshaping Human Reasoning and the Rise of Cognitive Surrender

Across three preregistered experiments (N = 1,372; 9,593 trials), they demonstrated that large majorities of people engage in ***“Cognitive Surrender”***

Cognitive Surrender

Core finding: **73.2% of participants accepted faulty AI reasoning**, while **only 19.7% overruled it**. When the AI was accurate, acceptance rose to 93%.

- One of the most consequential findings: participants who used AI scored **11.7% higher on self-reported confidence** in their answers, *even though the AI was deliberately setup to provide wrong answers half the time*.
- The researchers found that **time pressure**, the default condition of modern work, makes cognitive surrender *worse*, reducing override behavior by 12 percentage points.

Imaginal Agility Thesis: “AI is *eroding human metacognition at scale*, that this erosion is *invisible to those experiencing it*”

❖ “The danger wasn't AI. It was *deference to AI*”
i.e. *People do not recognize when they have stopped thinking*

Measuring AI Impact



Code Quality Erosion

AI gives the strong temptation to accept *plausible* generated code : [study](#)

- Only **48%** of developers *always review* AI-generated code before committing it
- **61%** of developers agree that AI often produces code that looks correct but isn't reliable
- **61%** of developers agree that it requires a lot of effort to get good code from AI
- **36%** of organizations are more rigorous about code quality because of AI

AI Generated Code Policy

Major Responses from Open Source to AI generated code

- **GNU Compiler collection** : ***Total prohibition*** on AI-generated patches to protect the project's legal integrity and the technical reliability of the world's most critical software infrastructure
- **Linux Kernel**: *Prioritizes human accountability regardless of the tools used*. A maintainer must fully understand and be able to defend every line of code they submit
- **Oracle**: Centralized GenAI policies for the Java ecosystem (OpenJDK) which heavily manages AI adoption from the top down - Corporate Model

Human Cost - Invisible Work

AI productivity is burning out your best engineers : By [Sreenivasa Reddy](#)

Juniors are shipping faster than ever. [Seniors are architecting](#) with less effort. But what about your mid-level engineers? *They are drowning.*

While the rest of us were celebrating [velocity](#), they were quietly absorbing work that never made it onto a sprint board. Fixing AI mistakes. Adding the context models missed. Making sure generated code would actually hold up in a regulated production environment. **None of it showed up in [metrics](#), performance reviews, or promotion conversations.**

Human Cost - Invisible Work

AI productivity is burning out your best engineers : By [Sreenivasa Reddy](#)

Juniors are shipping faster than ever. [Seniors are architecting](#) with less effort. But what about your mid-level engineers? *They are drowning.*

While the rest of us were celebrating [velocity](#), they were quietly absorbing work that never made it onto a sprint board. Fixing AI mistakes. Adding the context models missed. Making sure generated code would actually hold up in a regulated production environment. **None of it showed up in [metrics](#), performance reviews, or promotion conversations.**

Junior engineers feel productive from day one. Senior engineers go from whiteboard sketch to working prototype before lunch. Yet, somebody still has to catch the [security](#) gaps, compliance risks, and architectural conflicts buried in years of tribal knowledge.

In Summary

Software development has entered a new Age of AI with opportunities to improve business and engineering.

Acceleration is a real problem - Most other problems already existed but are now magnified.

Ensure you are not only going faster but in the right direction with defined processes.

A new problem has arisen - Cognitive surrender, handing the thinking over to the AI

The New Age

The Work Model :

Old model : Code generation = work, Code review = incidental untracked overhead

Age of AI model : Code generation = lesser work, Code review = tracked work

Reviews are ***no longer a silent side activity*** and should show up as real work/points on sprints

If code generation is cheap, verification/judgment is expensive

Additionally, time should be set aside for developers to experiment with AI while not under pressure to produce.

The New Age

When using AI for code generation with a ***high cost of being wrong***

- ☐ Quiz and challenge the AI on the code being generated and its architecture
- ☐ Ask the AI to point out potential shortcomings/trade-offs in its design/implementation
- ☐ Request alternatives
- ☐ Use the PR review AI as a help to understand potential issues when drafting the PR
- ☐ Stay in control and own final decisions

This keeps the developer engaged and able to understand what the code is doing and allows steering of the design/implementation - Mitigating cognitive debt

The New Age

Review hygiene :

- ☐ Adopt a “Linux” mentality, A PR author should be able to explain and defend every line of code they submit.
 - ☐ Pluck out an interesting piece and have them explain it.
- ☐ Ban review and approval comments of “LGTM”/blank
 - ☐ A brief explanation of why its passing review/approval for anybody to understand why
Checked X, I am comfortable with the change because...
- ☐ Review the tests, not just the fact that tests exist
 - ☐ What behavior is this test proving?
 - ☐ Black box state testing or testing hidden implementation

The New Age

Repository Guard rails:

- ☐ Repositories set up to prohibit the author from approving their own PR
- ☐ Enable CI consistently
- ☐ Deploy static-analysis checks
- ☐ Apply mechanical modernization aka clang-tidy
- ☐ Detect duplication aka PMD CPD
- ☐ Detect churn

The New Age

Measure outcomes:

- ☐ Time to merge
- ☐ Time to production
- ☐ Abandoned PRs
- ☐ No touch approvals
- ☐ PR size distribution
- ☐ Production incidents/Recovery time
- ☐ Serious defects revealed during review *

Public Service Announcement



**I'M BACK TO LOVING
AI AND NOW I HAVE A
FRAMEWORK THAT
MAKES SENSE**

Call To Action

AI is changing the world and harnessing this can be a good thing,
but we must adapt

Don't be a meat proxy!

(Someone who blindly copies, pastes, and forwards AI-generated text or code to others
without reading, understanding, or validating it first)

Forget about the token bragging

**Take control, be demanding and push AI to justify decisions it makes
that you will be attaching your name and reputation to**

The top corners of the slide are decorated with a spray of small, multi-colored dots in shades of green, blue, red, and purple, resembling confetti.

THE END FOR NOW

Thank You



Questions?

Contact: pmuldoon1@Bloomberg.net



Other Engineering Talks:

Retiring The Singleton Pattern: *Concrete Suggestions on What to Use Instead*

Redesigning Legacy Systems: *Keys to success* *

Managing External APIs in Enterprise Systems

Exceptions in C++: *Better Design Through Analysis of Real-World Usage*

Dependency Injection in C++: *A Practical Guide*

Mastering the Code Review Process: *Boosting Code Quality in your Organization* *

Software Engineering Pyramid: *Knowing when you are Done and why it matters* *

Modernizing Legacy Codebases Without Stopping the World

* Language agnostic

Thank you!

LinkedIn: [<https://www.linkedin.com/in/petemuldoon/>]

Bloomberg
Engineering

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



TechAtBloomberg.com

Engineering

Bloomberg